

# WSTĘP DO PROGRAMOWANIA

## Lista 1

1

2

3

### Zadanie 1

Rozważmy następujący kod:

```
1 #include <stdio.h>
2
3 #define MAGIC(x) sum += x * x
4
5 int main(void) {
6     int it, sum;
7     scanf("%d", &it);
8
9     for (unsigned int i = it; i >= 0; i++) {
10         MAGIC(i);
11     }
12
13     printf("%d\n", sum);
14 }
```

1. Cemu ten kod nie zadziała poprawnie?
2. Napraw ten kod.
3. Wyjaśnij, co robi `scanf` oraz czym jest `MAGIC` w tym kodzie. Czym różni się `MAGIC` od zwykłej funkcji?
4. Co oblicza ten kod?

**Wskazówka do podpunktu 3:** Makra.

### Zadanie 2

Nie korzystając z funkcji `printf` zaimplementuj rysowanie szachownicy o podanych parametrach, podanych jako argumenty wywołania.

- szerokość planszy (w „kratkach”),
- wysokość planszy (w „kratkach”),
- szerokość kratki,
- wysokość kratki,
- kolor kratki w (0, 0) (czarna/biała).

Czyli przykładowo wywołanie:

```
1 ./a.out 5 5 2 2 1
```

Powinno zakonczyc sie takim outputem:

```
1 ## ## ##
2 ## ## ##
3  ## ##
4  ## ##
5 ## ## ##
6 ## ## ##
7  ## ##
8  ## ##
9 ## ## ##
10 ## ## ##
```

**Wskazowka:** Wykorzystaj funkcje `putchar`. Aby pisac czytelny kod, uzywaj funkcji pomocniczych.

Pamietaj o sprawdzeniu wartosci argumentow na wejsciu!

### Zadanie 3

Lista jednokierunkowa to jedna z najprostszych struktur danych. Kazdy element takiej listy zawiera swoje faktyczne dane oraz wskaznik do kolejnego elementu. Latwy diagram ukazujacy ta strukture wygladalby nastepujaco:

```
Element 1 → Element 2 → Element 3 → Element 4
```

Mozemy zdefiniowac jeden element listy jednokierunkowej zawierajacej wartosci typu `int` w nastepujacy sposob:

```
1 typedef struct node {
2     int value;
3     struct node* next;
4 } Node;
```

Listy implementuja nastepujace funkcje, ktore na nich operuja:

- `push_back` / `push_front` - dodaje nowy element na koniec/poczatek listy,
- `pop_back` / `pop_front` - usuwa element z konca/poczatka listy,
- `new_list` - tworzy nowa, pusta liste (**0-elementowa!**)

Jezyk C nie posiada wbudowanych list. Zaimplementuj prosta liste jednokierunkowa przy uzyciu **wskaznikow**, nastepnie odpowiedz na pytania:

1. Jaki typ operacji `push` i `pop` bedzie szybszy - `_front` czy `_back`? Dlaczego?
2. Czemu mielibysmy uzywac listy zamiast tablic?
3. Jaka bedzie zlozonosc czasowa wszystkich operacji?